

ADL manual

S. Joosten

June 1, 2005

Abstract

The language ADL (A Description Language) allows you to specify patterns and contexts in relational algebra. This document contains the instructions for using the ADL language on your computer. We assume that you know how to use relational algebra for specifying patterns, for example by having studied the paper about rules, relations and contexts [1].

Keywords:

1 Introduction

ADL (A Description Language) is a computer language that allows you to define patterns, check patterns, use patterns in context, and generate useful texts and diagrams about them.

ADL comes as an application for Windows computers. It compiles ADL code and generates a web site. This is a collection of HTML documents ready to be published on a web server.

This tool is a valuable aid for experimenting with business rules. It gives feedback on syntax and type errors, allowing you to specify business rules correctly. It detects when and where the contents of your relations violate rules, allowing you to experiment with the meaning (semantics) of your rules. It translates your rules to predicat logic, allowing you to verify your rules from a different perspective. It draws (primitive) conceptual diagrams, illustrating the structure induced by your rules. Finally, it organizes rules, relations and concepts in a logical manner, allowing you to browse and explore various consequences of your ADL code.

Technically, ADL is heterogeneous relational algebra [2], drawing on an extensive body of available mathematical knowledge, some of which is well over a century of age. Practically, ADL is a simple, easy to learn language, whose structure fits well within three pages (section 5). People who have successfully completed their undergraduate courses in mathematics for computer science, including topics such as functions and relations, will see their efforts in learning ADL rewarded by successfully writing and using business rules for solving a variety of problems. For learning ADL, we refer to [1] and [2].

This paper is a brief ADL manual. It tells you how to install the tool (section 2), explains the technicalities of the language (section 3), explains error messages (section 4), and defines the syntax (section 5).

2 Installation

2.1 System Requirements

Please check the following requirements to see whether ADL will work properly on your system. You need Microsoft Windows with an Internet Explorer 4.0 or higher. Your Internet Explorer must be Javascript-enabled. When in doubt, check the internet options of your browser. On Windows-2000 and -XP you need administrator rights in order to install ADL.

2.2 Installing ADL

1. Unzip the file CC.zip in an empty directory, so you can see the contents. It consists of two *.exe files and six *.dll files. Besides, there is a directory called `treemenutils`, which contains a number of images and Javascripts.
2. Create a new directory, for example `C:\Program Files\ADL\bin\`, and move the files *.exe and *.dll into this directory. This directory now contains the executable programs.
3. Set the \$PATH variable to include `C:\Program Files\ADL\bin\`, or the directory of your own choosing in which you have put the executables. Now the program can be called from any working directory.
4. Make a new working directory for your ADL project, for example `C:\projects\tryADL\`.

5. Copy the directory `treemenutils` into this working directory¹. Now, inside `C:\projects\tryADL\` you have the directory `C:\projects\tryADL\treemenutils\`. Do not rename the directory `treemenutils`.
6. You are now ready to use ADL from within your project.

To test the installation, follow these steps:

1. Copy all files `*.adl` into your working directory, which we assume is `C:\projects\tryADL\`.
2. Open a new DOS-window. Right-click on the top bar of this window and choose **Properties** to enlarge the buffer. You can enlarge the buffer to allow for 9999 lines in your window, which is plenty. The standard size will cause too many error messages to scroll off the window, so we advise you to enlarge it.
3. Change the current directory in your DOS-window into `C:\projects\tryADL\`.
4. Call `ADL cd0`. This will analyze the file `cd0.adl` and return error messages if there are any. In this file on the distribution set there are none.
5. Now call `graphs`. This generates a number of `.png` files which contain conceptual diagrams.

If all went well, this is the dialogue you will see in your DOS-window:

```
C:> cd \projects\tryADL
C:> ADL cd0
Arguments: cd0
Switches:
cd0.adl is read.
cd0.adl has been parsed.
Concepts:
Object
Class
No type errors or cyclic specializations were found.
```

Generating Atlas for Metamodels in the current directory.

¹For the time being, you need a copy of this directory in every working directory you have. Also, you need one working directory per project. Yes, this creates a number of copies of the directory `treemenutils`. This is ugly, and a following version will undoubtedly do better. (Sorry for any inconvenience.)

(current directory must already contain the directory "treemenutils" with its complete contents)
index.html written, and Concept.html generated.
HTML code for context tree CTX_Metamodels.html written

Analyzing context Metamodels

HTML code for Metamodels navigator NAV_Metamodels.html written
HTML code for Class Diagram: Metamodels_Class_Diagram.html written
HTML code for Rule 1: Metamodels_Rule_1.html written
HTML code for concept Object: MetamodelsObject.html written
HTML code for navigator NAV_MetamodelsObject.html written
HTML code for concept Class: MetamodelsClass.html written
HTML code for navigator NAV_MetamodelsClass.html written
HTML code for REL_Metamodels_typeObjectClass.html written
HTML code for REL_Metamodels_isaClassClass.html written
HTML code for REL_Metamodels_classObjectClass.html written
C:> graphs

C:>neato -Tpng Metamodels.dot -o Metamodels.png

C:>neato -Tpng Metamodels_Class_Diagram.dot -o Metamodels_Class_Diagram.png

C:>neato -Tpng Metamodels_Rule_1.dot -o Metamodels_Rule_1.png

C:>neato -Tpng MetamodelsObject.dot -o MetamodelsObject.png

C:>neato -Tpng MetamodelsClass.dot -o MetamodelsClass.png

C:>

2.3 Viewing the result

The generated website is located in the working directory. You can view the file index.html from the working directory in the internet explorer.

3 Language

This section explains how rules, relations and concepts are written in ADL. We describe the main notions of the syntax. A full account of the syntax is given in section 5. By way of example, here is the text of a complete (but small) ADL file.

```

CONTEXT Metamodels
PATTERN "Objects"
CONCEPT Class "A description of a set of objects that share the same attributes, op
CONCEPT Object "An object is anything with a lifetime and a location." ""
CONCEPT Association "A connection between two classes which represents a relationsh

type :: Object * Class [UNI,TOT] PRAGMA "The class in which object " " was
isa :: Class * Class [RFX,TRN,ASY] PRAGMA "Every " " is " " as well". -- is a
class :: Object * Class [TOT] PRAGMA "Object " " has type ".
type;isa -:: class
EXPLANATION "Each object is a member of the class in which that object was instant
isa;isa -:: isa[Class*Class]
EXPLANATION "The relation isa is transitive."
ENDPATTERN

PATTERN "Class Diagram"
association :: Class * Class [] PRAGMA "" " is associated to ".
lineTo :: Association * Class [UNI,TOT] PRAGMA "Association " " links to "
lineFrom :: Association * Class [UNI,TOT] PRAGMA "Association " " links from "
name :: Association * String [UNI,TOT] PRAGMA "Association " " is named "
association = lineFrom~ ; lineTo
EXPLANATION "An association is represented by a line from one class to another"
ENDPATTERN

ENDCONTEXT

```

Since ADL is restricted to the characters on a computer keyboard, it uses character sequences instead of the original mathematical symbols. The following table shows the correspondence between ADL and mathematical symbols.

ADL	math	used in
-:	$\subseteq$	rules
=	$=$	rules
;	$;$	expressions
*	$\times$	declarations
$\setminus$	$\cup$	expressions
$\wedge$	$\cap$	expressions
~	$\sim$	expressions
I	$\mathbb{I}$	expressions

In ADL, the layout (i.e. placement of whitespace and newlines) is immaterial. You can use whitespace to structure your code to your own liking.

3.1 Comments

There are two different types of comment: nested comments and line comments. A nested comment is any text between comment brackets (`{-` and `-}`). Nested comments may be enclosed in other (larger) nested comments. A line comment is the character sequence following a double hyphen (`--`) until the next end of line. ADL ignores both types of comment.

3.2 Rule

In ADL, rules are defined within a *pattern*. The syntax of ADL is given in Backus Naur Form (BNF), for example²:

```
<rule> ::= <expression> "-:" <expression> "." .
<rule> ::= <relation> "=" <expression> "." .
```

You can read the first line as: a rule can be an expression followed by the `-:` symbol, followed by another expression and terminated by a dot (full stop symbol). Similarly, you read the second line as: a rule can be a relation followed by the `=` symbol, followed by an expression and terminated by a dot. An example of a rule is `elem;partof -: elem`, in which `elem;partof` is an expression and `elem` is an expression too.

3.3 Expression

The syntax of expressions is given by:

```
<expression> ::= <relation> .
<expression> ::= <relation> "~" .
<expression> ::= <expression> ";" <expression> .
<expression> ::= <expression> "/" <expression> .
<expression> ::= <expression> "\" <expression> .
<expression> ::= "(" <expression> ")" .
```

The postfix operator `~` binds stronger than any other operator, so the expression `r;s~` is different from `(r;s)~`. Of all three infix operators, `;` binds the strongest, followed by `/` and finally `\`. This means that `r/\s;p` is read `r/(s;p)` and `r/\s\p` is read `(r/\s)\p`.

²The examples given here are accurate, but not necessarily complete (for the sake of uncluttered explanations). Section 5 contains the correct syntax.

A relation mentioned in a rule refers to a relation declared somewhere in the context. For instance, the rule `elem;subset -: elem` refers to relations called `elem`, `subset`, and `elem`, which have their declarations elsewhere in the context. The source- and target concepts together are called the *signature* or *type* of a relation. ADL tries to deduce the signature of every relation. As ADL can often deduce it from the context, the name alone is often sufficient to identify a relation uniquely. To prevent ambiguities, you may write a relation in full, i.e. the name together with source- and target concepts as (for example) `elem[Atom*Set]`.

Take note of the fact that the same name may sometimes refer to different relations. If that name is shared by more than one relation. Remember that the combination of name, source and target identifies the relation; the name alone may not be enough. To prevent confusion, either avoid sharing names or disambiguate by writing a relation in full.

For each concept `C`, the relation `I[C]` is already defined. You do not need to declare `I[C]`, as you have to with other relations. If it is clear from the context that `I[C]` is meant, you may safely use `I`.

ADL is a case sensitive language. Identifiers of a relation always start with a lower case letter (`mIdent`), but names that identify concepts always start with a capital letter (`cIdent`). The syntax of relations is given by:

```
<relation> ::= mIdent .
<relation> ::= mIdent "[" cIdent "*" cIdent "]" .
<relation> ::= mIdent "[" cIdent "]" .
```

If you specify name and signature together, as in `president[Name*Country]`, you know for sure that this is the signature used by ADL. If you specify only one concept, as in `friend[Name]`, ADL interprets this as `friend[Name*Name]`. If you do not specify any concepts, ADL will try to find out which signature you mean.

3.4 Declaration

Every relation that you refer to in a rule must be declared first, so the compiler knows which properties you intend that relation to possess. Here is an example:

```
paid :: Client * Invoice.
```

This is the simplest notation. Notice the full stop (dot) at the end. It terminates the declaration and must be there for every declaration. You can insert multiplicity properties, pragmatics and contents (in that particular order) just before the full stop at the end of a declaration. The content is always the last item. Here is an example of a relation together with its contents:

```
paid :: Client * Invoice
  = [ ("Applegate", "5362a")
      ; ("Brown", "721i")
      ; ("Conway", "9443a")
      ].
```

If no content is defined, the relation is considered empty.

You can also specify the pragmatics of a relation by adding two or three fragments of sentences to your declaration, preceded by the word **PRAGMA**.

```
paid :: Client * Invoice
      PRAGMA "Client "
            " has paid invoice number "
            " in full"
  = [ ("Applegate", "5362a")
      ; ("Brown", "721i")
      ; ("Conway", "9443a")
      ].
```

This is your way of saying that client Applegate has paid invoice number 5362a in full, client Brown has paid invoice 721i in full, and client Conway has paid invoice 9443a in full. The **PRAGMA** has no consequence for the formal manipulations, but it tells you what meaning every link in your relation has in practice.

Finally, you can add the following relational properties to your declaration.

ADL	meaning	restriction
INJ	injective	
SUR	surjective	
UNI	univalent	
TOT	total	
RFX	reflexive	homogeneous relations only
TRN	transitive	homogeneous relations only
SYM	symmetric	homogeneous relations only
ASY	antisymmetric	homogeneous relations only

ADL uses these properties in reasoning about your rules. The first four properties (INJ, SUR, UNI, and TOT) are called multiplicities. The other four properties are applicable to homogeneous relations only. ADL produces an error message if you use them on a heterogeneous relation. Here is an example of declaring some properties

```
paid :: Client * Invoice [SUR,INJ]
    PRAGMA "Client "
        " has paid invoice number "
        " in full"
= [ ("Applegate", "5362a")
    ; ("Brown", "721i")
    ; ("Conway", "9443a")
    ].
```

This example declares `paid :: Client * Invoice` as a relation that is both surjective and injective. This means that every invoice has been paid in full by precisely one client. The ADL system signals all instances in which this property is not satisfied.

Computer professionals, who are familiar with UML, ERD or similar modeling techniques, have repeatedly asked for a table that relates the multiplicity properties (univalent, total, surjective and injective) to the notations they are used to. We hope they will find figure 1 useful.

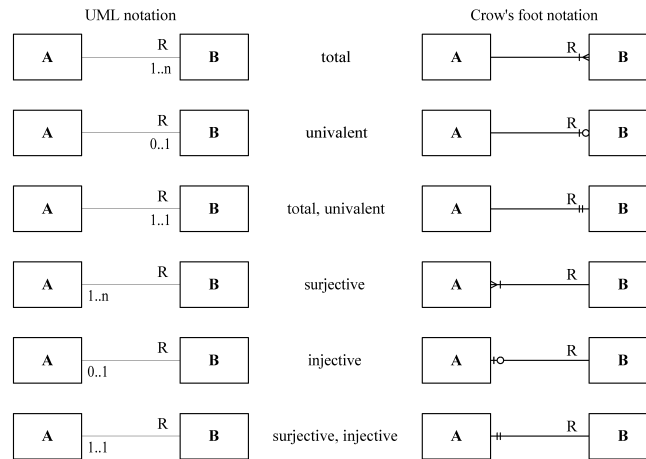


Figure 1: Multiplicities

The full syntax of the ADL language is given in section 5.

3.5 Contexts and Patterns

ADL uses the idea of patterns to collect a number of rules that belong logically together. When that pattern is used in a context, all rules defined in that pattern apply within the context. Within the context itself, extra rules may be defined for the purpose of glueing patterns together. So, all rules that apply in a context are the ones defined in patterns used by the context plus the rules defined within that context.

Your ADL file essentially describes the domain knowledge of a specific context. You have to provide a context name at the beginning. Keywords `CONTEXT` and `ENDCONTEXT` are used at the beginning and end of your file. Example:

```
CONTEXT Domain
```

write your patterns here

```
ENDCONTEXT
```

Inside the context you write patterns:

```
PATTERN Domain
```

write your rules, declarations and concept definitions here

```
ENDPATTERN
```

Every relation used in a pattern must be declared, though not necessarily inside the same pattern³. A rule may use any relation that is defined within the context, irrespective of which pattern it is declared in.

4 Mistakes

ADL signals many kinds of mistakes, meant to help you create flawless specifications. You need to distinguish between three kinds of mistake

- syntax errors

³This may change in future releases.

- type errors
- mistakes in calling the program ADL

Syntax errors can be recognized by an excessive amount of question marks. Examples:

```

????????????????????????????????????????????????????????????
?? Error      : before ";" at line 1295, column 28 of
??            file "mymodels.adl"
?? Expecting  : symbol ) or (symbol , ...)*
?? Repaired by: inserting symbol symbol )
????????????????????????????????????????????????????????????

????????????????????????????????????????????????????????????
?? Error      : before string "Ondernemend" at line 1295,
??            column 29 of file "mymodels.adl"
?? Expecting  : symbol (
?? Repaired by: inserting symbol symbol (
????????????????????????????????????????????????????????????

```

A single mistake can cause an avalanche of syntax errors, so here is what you need to do. Study the first message carefully. (You may have to scroll upwards in your window to find it.) Take note of the line and column number in the first error message and look at the repair action that ADL has carried out for you. Fix this problem and run ADL again immediately. You will find that many messages will disappear simultaneously by correcting a single syntax error. Repeat this procedure until you have corrected all syntax errors.

Besides signalling syntax errors, ADL repairs them as well. Consequently, ADL may generate output while there are remaining syntax errors. If ADL has repaired your mistakes in some unintended way, you might get unintended results. So you might want to check each time whether you have resolved all syntax errors.

The second type of mistakes are type errors. Such errors cause ADL to block. As long as type errors exist, output will not be generated. There are several kinds of type error, each of which may have different causes. We discuss each type of type error individually. Every type error carries a number identifying the type of error made, which corresponds to the following list.

4.1 Error: Mismatch in rule

ADL requires that signatures to the left and the right of the $-:$ or $=$ symbols are compatible. If not, the following error message occurs:

```
!Error of type 1 in line <line number>, file <file name>
Mismatch in rule:
<left hand side> :: <list of possible signatures>
does not match
<right hand side> :: <list of possible signatures>
```

Here is an example of this type of error:

```
!Error of type 1 in line 25, file "sick.adl"
Mismatch in rule:
r :: [C*C] or [A*A]
does not match
I[B] :: [B*B]
```

This message says that the left hand side and right hand side of the rule do not match. ADL checks all possible allocations of signatures, both for the left- and right hand sides.

Questions you may use for diagnosis:

- Make a drawing (conceptual diagram) for this specific rule. Are all relations in a cycle?
- Check whether the signatures that were intended by you (both for the left hand side and the right hand side of the rule) are mentioned in the error message.
- Are any relations missing to the left or the right of either the left hand side or right hand side of this rule?

If your answers to these questions did not expose your problem, and studying your code, error message and your diagram have not yielded a solution, you might carry out the following experiment. Add source- and target concepts between square brackets to precisely one of the relations in the erroneous rule. This limits the possibilities for ADL. You may start with one of the relations of which you are sure you know the signature. For every signature you add, run ADL immediately in order to study the effect on the error message. If this does not lead you to finding a solution, repeat this for the other relations in the rule.

4.2 Error: Ambiguous signatures

ADL requires all relations that are used in a rule are unambiguously determined. Since every relation is determined by its entire signature (name, source and target), ADL reconstructs the sources and targets of all relations used in a rule. An ambiguous rule results in the following error message

```
!Error of type 2 in line <line number>, file <file name>
  Ambiguous types: <list of signatures>
  in right hand side: <expression>
  and left hand side: <expression>
```

Here is an example of this type of error:

```
!Error of type 2 in line 27, file "sick.adl"
  Ambiguous types: [E*E] or [D*D]
  in right hand side: v
  and left hand side: u
```

The left hand side and right hand side of a rule can be matched in more than one way. The error message gives all possibilities. If several possibilities remain, this is signalled as an ambiguity. ADL requires a unique match.

Questions you may use for diagnosis:

- Make a drawing (conceptual diagram) for this rule only. Did you choose names for relations that occur in other patterns as well?
- Are names of relations sufficiently distinctive?
- Will the rule be more understandable by adding a type to one of the relations used in this rule?

Study the error message and compare it to your code and to your diagram for this particular rule. Your mistake can always be fixed by adding signatures to relations. However, sometimes it may be more appropriate to choose different names for different relations, thus resolving your ambiguity. If you add signatures to relations, this will limit the possibilities for ADL. For every signature you add, run ADL immediately in order to study the effect on the error message. After having added the signature of each relation explicitly, all ambiguity has been removed completely.

4.3 Error: Relation not declared

All relations that are used in any rule must be declared. If not, ADL generates the following message.

```
!Error of type 3 in line <line number>, file <file name>
  Relation <relation name> is not declared.
```

Here is an example of this type of error:

```
!Error of type 3 in line 33, file "myRules.adl"
  Relation sendr is not declared.
```

Your code contains no declaration of the relation mentioned in this message.

Questions you may use for diagnosis:

- Where did you define this relation?
- Is the spelling correct (case sensitive!)?

You must either correct (if misspelled) or define this relation.

4.4 Error: Incompatible comparison

In $\setminus/$ or $/\setminus$ expressions, signatures must be compatible. If not, ADL produces the following message:

```
!Error of type 4 in line <line number>, file <file name>
  Incompatible concepts in comparing
  <expression with list of possible signatures> with
  <expression with list of possible signatures>
```

Here is an example of this type of error:

```
!Error of type 4 in line 43, file "sick.adl"
  Incompatible concepts in comparing
  u :: [D*D] or [E*E] with
  r :: [C*C] or [A*A]
```

The signatures of the two expressions mentioned in this message are not compatible. The error message points out which expressions have caused the problem and gives their possible signatures.

Questions you may use for diagnosis:

- Make a drawing (conceptual diagram) for the two subexpressions mentioned in the error message. Are both expressions in a cycle?
- Are relations missing in order to complete the cycle?
- Are the signatures mentioned in the error message the ones you intended?

If your answers to these questions did not expose your problem, and studying your code, error message and your diagram have not yielded a solution, you might carry out the following experiment. Add a signature to one of the relations in the erroneous rule. This limits the possibilities for ADL. You may start with one of the relations of which you are sure you know the signature. For every signature you add, run ADL immediately in order to study the effect on the error message. If this does not lead you to finding a solution, repeat this for the other relations in the rule.

4.5 Error: Composition of relations

The combination operator ($;$), when used in an expression (e.g. $u;v$), requires that the target from its left argument (u) corresponds to the source of its right argument (v)

```
!Error of type 5 in line <line number>, file <file name>
  Type error in composing <expression with list of possible signatures>
  with <expression with list of possible signatures>
```

Here is an example of this type of error:

```
!Error of type 5 in line 55, file "sick.adl"
  Type error in composing
  u :: [D*D] or [E*E] with
  r :: [C*C] or [A*A]
```

Somewhere in the middle of an expression, you have used a combination operator (semicolon) where the relation left of it does not link to the relation on its right. The two concepts should be compatible. In the example, neither concept D nor E are compatible with D or A.

Questions you may use for diagnosis:

- In which rule does this occur? Locate the relations in your code. Find the semicolon with the problem.
- Are the relations in the error message the ones you intended (compare signatures)?
- The problem may be caused by a missing (or superfluous) FLP. Are they correct?

Drawing the conceptual diagram helps. Check the direction of the relations carefully, and check whether your FLP-operators are correct. You may experiment by adding signatures to the relations to the left and right of the semicolon. This limits the possibilities for ADL. You may start with one of the relations of which you are sure you know the signature. For every signature you add, run ADL immediately in order to study the effect on the error message. If this does not lead you to finding a solution, repeat this for the other relations in the rule.

4.6 Error: Composition of relations

The combination operator (`;`), when used in an expression (e.g. `u;v`), requires that the target from its left argument (`u`) corresponds to the source of its right argument (`v`)

```
!Error of type 6 in line <line number>, file <file name>
  Ambiguous composition of
  <expression with list of possible signatures> with
  <expression with list of possible signatures> over concepts <list of concepts>
```

Here is an example of this type of error:

```
!Error of type 6 in line 82, file "sick.adl"
  Ambiguous composition of
  p :: [M*S] or [W*T] with
  j~ :: [T*C] or [S*K] over concepts [S,T]
```

Somewhere in the middle of an expression, you have used a combination operator (semicolon) where the relation left of it may link to the relation on its right in different ways. This is signalled as an ambiguity. In the example, either concept `S` and `T` might serve to link the two subexpressions.

Questions you may use for diagnosis:

- In which rule does this occur? Locate the relations in your code. Find the semicolon with the problem.
- Are the relations in the error message the ones you intended (compare signatures)?
- Could you be more specific and narrow the signatures?

Drawing the conceptual diagram helps. Check the direction of the relations carefully, and check whether your FLP-operators are correct. You may experiment by adding signatures to the relations to the left and right of the semicolon. This limits the possibilities for ADL. You may start with one of the relations of which you are sure you know the signature. For every signature you add, run ADL immediately in order to study the effect on the error message. If this does not lead you to finding a solution, repeat this for the other relations in the rule.

4.7 Error: Too many concepts

Relations can be restricted by explicitly mentioning two concepts between square brackets. This allows you to specify precisely one relation. If you provide more than two concepts, you will get the following error message.

```
!Error of type 7 in line <line number>, file <file name>
  Relation <relation>
  is restricted by more than two concepts.
```

Here is an example of this type of error:

```
!Error of type 7 in line 84, file "sick.adl"
  Relation s
  has more than two concepts.
```

Rethink this relation and restrict the list to two concepts.

4.8 Error: Properties

The properties transitive (TRN), reflexive (RFX), symmetric (SYM) and antisymmetric (ASY) are reserved for homogeneous relations. If you specify them for a heterogeneous relation, you will get the following error message.

```
!Error of type 9 in line <line number>, file <file name>
  Heterogeneous relation <relation with signature>
  may not be declared <properties>.
  (This relation is heterogeneous because <Concept> and <Concept>
   are different concepts).
```

Here is an example of this type of error:

```
!Error of type 9 in line 56, file "myRules.adl"
  Heterogeneous relation roeptaan[Component*Service]
  may not be declared transitive and reflexive.
  (This relation is heterogeneous because Component and Service
   are different concepts).
```

A relation is homogeneous if its source and target are the same. Otherwise, the relation is called heterogeneous. For each occurrence of this error message, your file contains a relation with different source and target concepts, which is specified to be transitive, reflexive, symmetric or antisymmetric.

Questions you may use for diagnosis:

- Did you intend this relation to be heterogeneous?
- If you meant it to be homogeneous, is there a spelling error in the signature that explains the message?
- If you meant it to be heterogeneous, why did you specify TNR, RFX, SYM or ASY in the property list?

If you meant the relation to be homogeneous, correct the signature and make source and targets identical (case sensitive!) If you meant the relation to be heterogeneous, correct the property list and remove all occurrences of TNR, RFX, SYM and ASY.

This concludes the list of possible type errors.

4.9 ADL errors

The last category of errors are caused by mistakes in calling the program ADL. ADL expects you to say in which file you have described your contexts.

Please provide a filename (.adl) and a context name.

Apparently, you have called the program ADL without specifying an argument. ADL expects the name of a .adl-file (such as `filename.adl`). By the way, you may leave out the extension (`.adl`).

Here are some examples of calling the program:

```
C> ADL mymodels.adl
C> ADL mymodels
C> ADL mymodels Petrinet
```

In your file you may specify more than one context. If you do so, ADL expects you to state the name of one context as the second argument. The example shows `Petrinet` in this position. Failure to do so or specifying a name that is not an available context yields the following error message:

```
Please specify the name of a context.
Available contexts: <list of context names>.
```

The following example illustrates this message:

```
Please specify the name of a context.
Available contexts: Skillbased and DocMan.
```

If you get such a message, your .adl file contains more than one context. ADL requires you to specify the context to be analyzed in the second argument on the command line. If you specified a second argument, check its spelling. All possible choices you have are given in this message.

5 Syntax

This section describes the syntax of ADL for reference purposes only.

5.1 Tokens

ADL is a case sensitive language. There are two different types of comment: nested comments and line comments. A nested comment is any text between comment brackets (`{-` and `-}`). Nested comments may be enclosed in other (larger) nested comments. A line comment is the character sequence following a double hyphen (`--`) until the next end of line. ADL ignores both types of comment.

Tokens (also known as 'terminal symbols') are identified from the input, and subsequently parsed by ADL. The language distinguishes between keywords, special symbols, strings and identifiers, all of which are considered tokens.

ADL contains the following keywords:

ASY	CONCEPT	CONTEXT	ENDCONTEXT	ENDPATTERN
ENDPOPULATION	EXPLANATION	GEN	GLUE	I
IMPORT	INJ	ISA	PATTERN	POPULATION
PRAGMA	RELATION	RFX	SUR	SYM
TOT	TRN	UNI		

The following special symbols are used in ADL:

-: = ; * :: \ / \ () [] . ,

ADL has two types of identifiers: A **C_identifier** begins with a capital letter and an **L_identifier** begins with a lower case letter. A **String** is a sequence of characters enclosed by double quotes, with quotes themselves escaped by a backslash.

The nonterminal symbols of ADL are enclosed in sharp brackets. The start symbol of ADL is **<contexts>**. The metasyntax of the production rules reads (in ascending order of precedence):

n ::= e . nonterminal **n** is defined by expression **e**
[e] optionally expression **e**
n | m either expression **n** or expression **m**
n ; m expression **n** followed by expression **m**
n+tok nonterminal **n** repeated once or more, separated by token **tok**
n+ nonterminal **n** repeated once or more
n*tok any number of nonterminals **n**, separated by token **tok**
n* any number of nonterminals **n**

The production rules of ADL are listed below.

```

<context>      ::= "CONTEXT" ; C_identifier          ;
                  <contextElement>*                  ;
                  "ENDCONTEXT" .

<contextElement>
  ::= <pattern> | <morphism> | <conceptDef> .

<pattern>
  ::= "PATTERN" ; <concept> ; <patElem>* ; "ENDPATTERN" .

```

```

<concept>      ::= C_identifier | String .

<patElem>
  ::= <rule> | <csubs> | <gen> | <morphism> | <conceptDef> .

<rule>
  ::= <factorU> ; "-" ; <factorU> ; [ "EXPLANATION" ; String ] |
     <morph> ; "=" ; <factorU> ; [ "EXPLANATION" ; String ] .

<csubs>        ::= "GLUE" ; <morph> ; "=" ; <factorU> .

<gen>          ::= "GEN" ; <concept> ; "ISA" ; <concept> .

<term>         ::= <morph>
                  |
                  <morph> ; "~"
                  |
                  "(" ; <factorU> ; ")"
                  |
                  "(" ; <factorU> ; ")" ; "~" .

<factorU>      ::= <factorI>+"\"/" .

<factorI>      ::= <factor>+"\"/" .

<factor>       ::= <term>+";" .

<morph>        ::= L_identifier ; [ "[" ; <concept>*"*" ; "]" ] .

<conceptDef>   ::= "CONCEPT" ; <concept> ; String ; String .

<morphism>
  ::= L_identifier ; ":" ; <concept> ; "*" ; <concept> ;
     [ <props> ] ;
     [ <pragma> ] ;
     [ <content> ] ;
     "." .

<content>      ::= "=" ; "[" ; <record>*";" ; "]" .

<props>        ::= "[" ; <prop>*""," ; "]" .

<prop>         ::= "UNI" | "INJ" | "SUR" | "TOT" |
                  "SYM" | "ASY" | "TRN" | "RFX" .

<pragma>       ::= "PRAGMA" ; String ; String
                  |
                  "PRAGMA" ; String ; String ; String .

```

`<record> ::= "(" ; String ; "," ; String ; ")" .`

References

- [1] JOOSTEN, S. Rules, relations and concepts. Paper written for the course Methods of Information Systems Development, 2005.
- [2] SCHMIDT, G., HATTENSPERGER, C., AND WINTER, M. *Heterogeneous Relation Algebra*. in: Relational Methods in Computer Science, Advances in Computing Science. Springer-Verlag, 1997, ch. 3, pp. 39–53.